

6. Appendix

A. ESP32 CODE

```
#include <WiFi.h>

#include <HTTPClient.h>

#include <WiFiClientSecure.h>

const char* ssid    = "test"; const char* password = "123456789"; const char* apiUrl
= "https://adstnqux35or2vfzoz2zpquoka0vwfkc.lambda-url.us-east-
1.on.aws/";

#define RXD2 16
#define TXD2 17

bool flameAlertSent    = false; bool
suppressedAlertSent = false;

void sendAlert(String type) {
  WiFiClientSecure client;
  client.setInsecure(); HTTPClient
http; http.begin(client, apiUrl);
  http.addHeader("Content-Type", "application/json");
  String body = "{\"type\":\"" + type + "\"}"; int code =
  http.POST(body);
  Serial.println("HTTP: " + String(code) + " type: " + type); http.end();
}
```

```
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
  WiFi.begin(ssid, password);
  Serial.print("Connecting WiFi"); while
(WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}
  Serial.println("\nWiFi Connected!");
}

void loop() {      if
(Serial2.available()) {
  String  msg  =  Serial2.readStringUntil('\n');
  msg.trim();
  Serial.println("From Arduino: " + msg);

  if ((msg == "SPRAY_RIGHT" ||          msg ==
"SPRAY_LEFT" ||          msg == "SPRAY_FRONT" ||
msg == "MOVE_RIGHT" ||          msg == "MOVE_LEFT"
||          msg == "MOVE_FORWARD") && !flameAlertSent)
{  sendAlert("flame");  flameAlertSent  = true;
  suppressedAlertSent = false;
}
```

```
    if (msg == "NO_FIRE" && flameAlertSent && !suppressedAlertSent) {  
    sendAlert("suppressed");    suppressedAlertSent = true;  
    flameAlertSent    = false;  
    }  
    }  
}
```

B. Arduino CODE

```
#include <Servo.h>
```

```
// ===== MOTOR DRIVER =====  
const int ENA = 5; const int IN1 = 13; const int IN2 = 11; const int  
IN3 = 10; const int IN4 = 9; const int ENB = 6;  
// ===== FLAME SENSORS =====  
const int RIGHT_SENSOR = A0; const int FRONT_SENSOR = A1;  
const int LEFT_SENSOR = A3;  
// ===== OUTPUTS =====  
const int BUZZER = 8; const int PUMP = 7;  
// ===== SERVO =====  
Servo myServo; const int SERVO_PIN = 4;  
// ===== MOTOR SPEED ===== const  
int motorSpeed = 150;  
  
// baseline no fire readings  
const int R_BASE = 1000;  
const int F_BASE = 480; const  
int L_BASE = 1000;  
  
// detect far fire — small drop = fire  
const int fireThresh = 50;  
// stop at safe distance — bigger drop = near const  
int nearThresh = 150;  
  
void setup() { pinMode(ENA,  
OUTPUT); pinMode(IN1,  
OUTPUT); pinMode(IN2,  
OUTPUT); pinMode(IN3,  
OUTPUT); pinMode(IN4,
```

```
OUTPUT); pinMode(ENB,  
OUTPUT); pinMode(BUZZER,  
OUTPUT); pinMode(PUMP,  
OUTPUT);  
digitalWrite(BUZZER, LOW);  
digitalWrite(PUMP, LOW);  
myServo.attach(SERVO_PIN);  
myServo.write(90);  
Serial.begin(9600); stopCar();  
}
```

```
void loop() {  
    int R = analogRead(RIGHT_SENSOR);  
    int F = analogRead(FRONT_SENSOR);  
    int L = analogRead(LEFT_SENSOR);
```

```
    int Rdrop = R_BASE - R;  
    int Fdrop = F_BASE - F;    int  
    Ldrop = L_BASE - L;
```

```
    bool Rfire = Rdrop > fireThresh;  
    bool Ffire = Fdrop > fireThresh;    bool  
    Lfire = Ldrop > fireThresh;
```

```
    bool Rnear = Rdrop > nearThresh;  
    bool Fnear = Fdrop > nearThresh;    bool  
    Lnear = Ldrop > nearThresh;
```

```

Serial.print("R:"); Serial.print(R);
Serial.print(" F:"); Serial.print(F);
Serial.print(" L:"); Serial.print(L);
Serial.print(" | Rd:"); Serial.print(Rdrop);
Serial.print(" Fd:"); Serial.print(Fdrop);
Serial.print(" Ld:"); Serial.println(Ldrop);

// =====
// NO FIRE
// =====
if (!Rfire && !Ffire && !Lfire) {  stopCar();  digitalWrite(PUMP,
LOW);  digitalWrite(BUZZER, LOW);  myServo.write(90);
    Serial.println("NO_FIRE");
}

// =====
// RIGHT FIRE
// =====
else if (Rfire && Rdrop >= Fdrop && Rdrop >= Ldrop) {
digitalWrite(BUZZER, HIGH);  if (Rnear) {
    // near — stop immediately and spray
stopCar();  myServo.write(45);
digitalWrite(PUMP, HIGH);
    Serial.println("SPRAY_RIGHT");
} else {
    // far — move toward fire
myServo.write(45);  // aim while moving
digitalWrite(PUMP, LOW);  turnRight();

```

```
    Serial.println("MOVE_RIGHT");
}
}

// =====
// LEFT FIRE
// =====

else if (Lfire && Ldrop >= Fdrop && Ldrop >= Rdrop) {
digitalWrite(BUZZER, HIGH);
    if (Lnear) {
stopCar();
myServo.write(135);
digitalWrite(PUMP,
HIGH);

        Serial.println("SPRAY_LEFT");
    } else {    myServo.write(135);    // aim
while moving    digitalWrite(PUMP, LOW);
turnLeft();
        Serial.println("MOVE_LEFT");
    }
}

// =====
// FRONT FIRE
// =====

else if (Ffire) {
    digitalWrite(BUZZER, HIGH);
if (Fnear) {    stopCar();
```

```
myServo.write(90);
digitalWrite(PUMP, HIGH);
    Serial.println("SPRAY_FRONT");
    } else {    myServo.write(90);    // aim
while moving    digitalWrite(PUMP, LOW);
moveForward();
    Serial.println("MOVE_FORWARD");
    }
}

else {    stopCar();
digitalWrite(PUMP, LOW);
digitalWrite(BUZZER, LOW);
myServo.write(90);
    Serial.println("NO_FIRE");
    }

    delay(500);
}

// ===== FORWARD =====
void moveForward() {    analogWrite(ENA, motorSpeed);
analogWrite(ENB, motorSpeed);    digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW);    digitalWrite(IN3, LOW);
digitalWrite(IN4, HIGH);
}

// ===== RIGHT =====
```



```
void turnRight() {
  analogWrite(ENA, motorSpeed);
  analogWrite(ENB, motorSpeed);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}

// ===== LEFT =====

void turnLeft() {
  analogWrite(ENA, motorSpeed);
  analogWrite(ENB, motorSpeed);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}

// ===== STOP =====

void stopCar() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}
```